

APOSTILA

Programação Orientada a Objetos em Java

Material de estudo, revisão e consulta

Modelagem

classes, objetos, encapsulamento

Estruturas

herança, interfaces, coleções

Execução

GUI, threads, streams e sockets

Programação Orientada a Objetos

Conceitos, padrões de código e exercícios resolvidos

Créditos: **Mateus Rodrigues**

Java
POO

Consulta de emergência I - "o enunciado pede..."

Sinal no enunciado	Estrutura mais provável	Consulte
Objetos diferentes oferecem a mesma operação	interface + classes com implements; variável do tipo da interface	p. 6
Há estado e implementação comuns, mas a base não deve ser instanciada	abstract class; métodos concretos e abstratos	p. 6
"É um tipo de"	Herança com extends; confirme substituição verdadeira	p. 6
"Tem", "possui" ou "usa" outro objeto	Composição: campo que referencia o colaborador	p. 6
Impedir preço, estoque ou quantidade inválidos	Campos private; validação no construtor e em métodos que alteram estado	p. 5
Mesmo nome, parâmetros diferentes	Sobrecarga: escolha feita na compilação	p. 5
Subclasse muda comportamento herdado	Sobrescrita com @Override: escolha em execução	p. 6
Quantidade dinâmica e acesso por índice	ArrayList<T> ou Vector<T> quando o exercício exigir	p. 8
Não aceitar duplicados	Set<T>; ordenado: TreeSet<T>	p. 8
Associar chave a valor ou contar frequências	Map<K, V>; ordenado por chave: TreeMap<K, V>	p. 8
Ordenar objetos	Collections.sort; Comparable OU Comparator	p. 9
Remover enquanto percorre	Iterator.remove(); forma Java 8: removeIf	p. 9
Reagir a botão, Enter ou item de menu	ActionListener: implementar, registrar e tratar	p. 10
Organizar uma janela	BorderLayout, GridLayout, FlowLayout e painéis aninhados	p. 9
Executar tarefas concorrentes	Herança de Thread ou implementação de Runnable; chamar start()	p. 10
Esperar threads antes de usar o resultado	Iniciar todas e depois chamar join() em todas	p. 11
Proteger dado compartilhado	synchronized; coordenação por wait/notifyAll	p. 11
Atualizar Swing a partir de worker	SwingUtilities.invokeLater(...)	p. 11
Comunicação cliente-servidor	Servidor: ServerSocket.accept; cliente: new Socket(host, porta)	p. 12

Decisão rápida - três relações que organizam a modelagem

Pergunta	Relação	Forma em Java
Todo B pode substituir A?	"é um" / especialização	class B extends A
A guarda ou usa B?	"tem um" / composição	campo private B b;
Classes diferentes devem saber fazer X?	contrato comum	interface X + implements X

Consulta de emergência II - esqueletos para adaptar no papel

Classe com estado válido

```
class Produto {
    private double preco;
    private int estoque;

    Produto(double preco, int estoque) {
        setPreco(preco);
        if (estoque < 0)
            throw new IllegalArgumentException();
        this.estoque = estoque;
    }

    void setPreco(double preco) {
        if (preco < 0)
            throw new IllegalArgumentException();
        this.preco = preco;
    }
}
```

Interface + polimorfismo

```
interface Operacao {
    double calcular(double a, double b);
}

class Soma implements Operacao {
    @Override public double calcular(double a, double b)
    {
        return a + b;
    }
}

Operacao op = new Soma();
double r = op.calcular(2, 3);
```

Entrada numérica sem encerrar

```
while (true) {
    try {
        int n = Integer.parseInt(sc.nextLine());
        break;
    } catch (NumberFormatException e) {
        System.out.println("Valor inválido");
    }
}
```

Coleções: buscar, contar, remover

```
Aluno achado = null;
for (Aluno a : alunos)
    if (a.getRa().equals(ra)) achado = a;

freq.put(nome, freq.getOrDefault(nome, 0) + 1);

Iterator<Aluno> it = alunos.iterator();
while (it.hasNext())
    if (it.next().getMedia() < 5) it.remove();
```

Listener em três passos

```
class Janela extends JFrame
    implements ActionListener {
    JButton b = new JButton("OK");
    Janela() {
        add(b);
        b.addActionListener(this); // registra
    }
    public void actionPerformed(ActionEvent e) {
        if (e.getSource() == b) { /* reação */ }
    }
}
```

Thread com Runnable + espera

```
class Tarefa implements Runnable {
    @Override public void run() { /* trabalho */ }
}

Thread t = new Thread(new Tarefa());
t.start();
// trabalho concorrente...
t.join(); // só continua quando t terminar
```

Atualização segura de Swing

```
// dentro da thread de trabalho:
final int v = valor;
SwingUtilities.invokeLater(new Runnable() {
    public void run() {
        barra.setValue(v);
    }
});
```

Socket: ordem mínima

```
// servidor
ServerSocket ss = new ServerSocket(5000);
Socket s = ss.accept();

// cliente
Socket s = new Socket("host", 5000);

// nos dois lados
InputStream in = s.getInputStream();
OutputStream out = s.getOutputStream();
```

Consulta de emergência III - análise de código e pegadinhas

Ao ver...	Pergunte...	Resultado ou regra
TipoBase x = new Subclasse()	Qual é o tipo declarado e qual é o objeto real?	O tipo declarado limita os membros acessíveis; a classe real escolhe a versão sobrescrita do método.
Dois métodos com o mesmo nome	Os parâmetros diferem? Há herança?	Parâmetros diferentes: sobrecarga. Mesma assinatura na subclasse: sobrescrita.
Campo ou método static	Depende de objeto?	Há uma cópia por classe. Método estático não acessa campo de instância sem receber ou criar um objeto.
== entre objetos ou Strings	Quer identidade ou conteúdo?	== compara referências; equals compara conteúdo conforme a classe.
for (T x : lista) + lista.remove(x)	A coleção é alterada pelo iterador correto?	Pode ocorrer ConcurrentModificationException; use Iterator.remove().
TreeSet com Comparator	O comparador retorna zero para objetos diferentes?	Zero significa equivalência para o conjunto; um elemento pode ser descartado.
Vários add na mesma região do BorderLayout	Há mais de um componente na região?	O último substitui o anterior; agrupe componentes em um JPanel.
new Thread(...).run()	Foi chamado start()?	run() direto é chamada comum na thread atual; não cria concorrência.
Saída de duas threads	Existe join, lock ou protocolo de coordenação?	A ordem é não determinística. sleep não estabelece ordem.
synchronized + laço vazio es- perando condição	O monitor é liberado?	Não. wait() suspende e libera o monitor; use dentro de synchronized.
Código demorado em actionPerformed	Está bloqueando a EDT?	A GUI congela. Execute o trabalho fora da EDT e publique apenas a atualização.
catch(Exception) antes do específico	O segundo catch ainda é alcançável?	Não; capture da subclasse específica para a superclasse geral.
Cliente e servidor chamam readLine() primeiro	Quem envia a primeira mensagem?	Os dois podem bloquear. Escreva o protocolo em duas colunas e alinhe envio e leitura.

Decisão rápida - roteiro de 45 segundos

1) marque classes, interfaces e relações; 2) separe tipo declarado do objeto real; 3) anote o estado alterado; 4) siga as chamadas na ordem efetiva; 5) marque pontos que podem lançar exceção; 6) em coleções, confira ordem, duplicidade e remoção; 7) em threads, pergunte quem executa, quem espera e qual dado é compartilhado; 8) em I/O e rede, alinhe recursos, protocolo e fechamento.

Distinções que costumam decidir a questão

Par	Primeiro	Segundo
overload X override	parâmetros diferentes; compilação	mesma assinatura na subclasse; execução
classe abstrata X interface	estado e implementação comum	contrato; múltiplas capacidades
throw X throws	cria/lança a exceção	declara possível propagação
start() X run()	inicia nova thread	chamada comum na atual

Índice

Os títulos usam os nomes pelos quais as dúvidas aparecem na prova. As referências das páginas são geradas automaticamente.

Consulta de emergência	1	5.2	Layouts e painéis aninhados	9
1 Classes, objetos, métodos e encapsulamento	5	5.3	pack(), fechamento e AWT x Swing	10
1.1 Classe, objeto, instância e membros	5	5.4	O que é um listener e os três passos obrigatórios	10
1.2 Membros de instância x membros static	5	5.5	Escolha do listener: evento semântico primeiro	10
1.3 Construtores e this	5	5.6	Listener x Adapter e classes internas	10
1.4 Sobrecarga (overload): mesmo nome, parâmetros diferentes	5	5.7	Um listener para vários componentes	10
1.5 Encapsulamento: controlar acesso e preservar invariantes	5	5.8	Como analisar um programa de GUI	10
1.6 Getters, setters, toString, igualdade e identidade	5	6 Threads, sincronização e Swing	10	
1.7 Passagem de parâmetros em Java	5	6.1	O que é uma thread e duas formas de definir a tarefa	10
1.8 Modificadores mais relevantes	5	6.2	sleep e ordem de execução	10
2 Relações entre classes, abstração e polimorfismo	6	6.3	Esperar conclusão com join	11
2.1 Composição: um objeto tem ou usa outro	6	6.4	Soma de matrizes por regiões exclusivas	11
2.2 Herança: especializar um tipo existente	6	6.5	Condição de corrida e operação atômica	11
2.3 Sobrescrita (override) e despacho dinâmico	6	6.6	Blocos sincronizados e granularidade	11
2.4 Polimorfismo: um contrato, várias implementações	6	6.7	Produtor-consumidor: wait e notifyAll	11
2.5 Classe abstrata: base incompleta com estado comum	6	6.8	Regra da thread única do Swing (EDT)	11
2.6 Interface: contrato entre tipos diferentes	6	6.9	invokeLater X invokeAndWait	12
2.7 Classe abstrata x interface	7	6.10	Timer Swing	12
2.8 Exemplo central: FormaGeometrica	7	7 Streams, arquivos e sockets	12	
3 Exceções: detectar, lançar, propagar e recuperar	7	7.1	O que é um stream e como escolher	12
3.1 O que é uma exceção e como o fluxo muda	7	7.2	Bytes, caracteres e buffer	12
3.2 try, catch e finally	7	7.3	Scanner, tokens e delimitadores	12
3.3 Checked x unchecked	7	7.4	Objetos, serialização e a classe File	12
3.4 throw X throws	7	7.5	Sockets: papéis e conexão TCP	12
3.5 Exceção personalizada	7	7.6	Protocolo: ordem e formato das mensagens	12
3.6 Entrada inválida sem encerrar o programa	8	8 Exercícios de implementação resolvidos	13	
3.7 Recursos e fechamento	8	8.1	Lista de revisão - exercícios 2 a 6	13
4 Collections, generics, iteração e ordenação	8	8.2	Lista 3 - Sistema de Delivery	17
4.1 O que é uma coleção e como escolher	8	8.3	Lista 4 - Exceções - exercícios 1 e 2	17
4.2 List: ArrayList, Vector e LinkedList	8	8.4	Lista 5 - Collections - exercício 3	18
4.3 Set: unicidade e ordem	8	8.5	Lista 6 - GUI - exercícios 1 a 4	18
4.4 Map: busca por chave e contagem	8	8.6	Lista 7 - Listeners - exercício 1	21
4.5 Fila e pilha	8	8.7	Lista 8 - Threads - exercícios 2, 6 e 7	21
4.6 Percorrer: for-each, Iterator e Enumeration	9	8.8	Lista 11 - Socket: jogo da velha em rede	23
4.7 Remoção durante a iteração	9	9 Checklist, resumo e índice reverso	27	
4.8 Ordem natural e Comparator	9	9.1	Checklist antes de entregar uma resposta de código	27
4.9 Algoritmos da classe Collections	9	9.2	Resumo em uma frase por assunto	27
5 Swing, layouts, eventos e listeners	9	9.3	Índice reverso por dúvida ou palavra-chave	28
5.1 Estrutura mínima de uma janela Swing	9			

Decisão rápida - duas formas de navegar

Por assunto: use este índice para localizar a definição e o exemplo principal. **Por problema:** use o índice reverso nas páginas finais para partir de sintomas como “GUI congelada”, “RA repetido” ou “método não sobrescreve”.

1 Classes, objetos, métodos e encapsulamento

Procure aqui: classe; objeto; instância; campo; método; static; construtor; this; sobrecarga; encapsulamento; invariante

1.1 Classe, objeto, instância e membros

Definição direta - classe e objeto

Uma **classe** é o molde que declara estado e comportamento comuns. Um **objeto** ou **instância** é uma ocorrência concreta dessa classe, normalmente criada com `new`. Campos guardam estado; métodos implementam comportamento.

Declaração, instanciação e mensagem

```
class NumeroComplexo {
    double real, imag;           // campos
    void atribuir(double r, double i) { // método
        real = r;
        imag = i;
    }
}
NumeroComplexo x = new NumeroComplexo();
x.atribuir(1.2, 9.8); // mensagem ao objeto x
```

1.2 Membros de instância x membros static

Cada objeto tem sua cópia dos campos de instância. Um membro `static` pertence à classe e existe em uma única cópia compartilhada.

	Instância	Classe (static)
Quantidade	uma por objeto	uma por classe
Acesso típico	obj.membro	Classe.membro
Método acessa	instância e estático	só estático diretamente

Erro frequente - contexto estático

main é static. Para acessar um campo de instância, ele precisa de um objeto: `Produto p = new Produto(...); p.vender(1);`. A mensagem "non-static variable ... from static context" indica que esse objeto faltou.

1.3 Construtores e this

Definição direta - construtor

Construtor é o bloco executado na criação do objeto. Tem exatamente o nome da classe, não possui tipo de retorno e deve terminar com o objeto em estado válido.

Construtores sobrecarregados

```
class Vetor2D {
    private float x, y;

    Vetor2D() {
        this(0, 0);           // chama outro construtor
    }
    Vetor2D(float x, float y) {
        this.x = x;           // campo recebe parâmetro
        this.y = y;
    }
}
```

Atenção - pegadinhas

- `void Caneta(...)` é método comum, não construtor.
- Se nenhum construtor é declarado, Java fornece um vazio implícito. Declarar qualquer construtor elimina esse implícito.
- `this(...)` ou `super(...)` deve ser a primeira instrução do construtor.

1.4 Sobrecarga (overload): mesmo nome, parâmetros diferentes

Definição direta - sobrecarga

Sobrecarga é declarar métodos ou construtores com o mesmo nome e listas de parâmetros diferentes em quantidade, tipos ou ordem. O compilador escolhe a versão pela chamada; alterar apenas o retorno não cria sobrecarga.

Duas formas de calcular produto escalar

```
float produtoEscalar(Vetor2D outro) {
    return x * outro.x + y * outro.y;
}
float produtoEscalar(float outroX, float outroY) {
    return x * outroX + y * outroY;
}
```

1.5 Encapsulamento: controlar acesso e preservar invariantes

Definição direta - encapsulamento

Encapsulamento é manter o estado e as regras de manipulação dentro da classe, escondendo alterações arbitrárias e expondo operações controladas. Ele serve para reduzir dependências e garantir que o objeto permaneça válido.

Uma **invariante** é uma condição que deve ser verdadeira após o construtor e continuar verdadeira depois de toda operação pública. Exemplos: preço e estoque não negativos; venda positiva e não maior que o estoque.

Produto: comportamento em vez de acesso direto

```
class Produto {
    private double preco;
    private int estoque;

    Produto(double preco, int estoque) {
        setPreco(preco);
        if (estoque < 0)
            throw new IllegalArgumentException("Estoque negativo");
        this.estoque = estoque;
    }
    void setPreco(double preco) {
        if (preco < 0)
            throw new IllegalArgumentException("Preço negativo");
        this.preco = preco;
    }
    void vender(int q) {
        if (q <= 0 || q > estoque)
            throw new IllegalArgumentException("Venda inválida");
        estoque -= q;
    }
}
```

Padrão seguro - onde validar

Valide no construtor e em todo método que altera o estado. Validar apenas na interface é frágil: outro chamador poderia contornar a regra. Faça a validação antes de qualquer alteração para evitar estado parcialmente modificado.

1.6 Getters, setters, toString, igualdade e identidade

Getter devolve um valor; setter só é útil quando a alteração é permitida e pode ser validada. Nem todo campo precisa de setter. `toString()` produz representação textual útil para impressão e análise.

Acesso controlado e representação

```
String getNome() { return nome; }
void setPreco(double preco) {
    if (preco < 0) throw new IllegalArgumentException();
    this.preco = preco;
}
@Override public String toString() {
    return nome + " - R$ " + preco + " - " + estoque + " un.";
}
```

`==` testa se duas variáveis apontam para o mesmo objeto. `equals` testa conteúdo conforme a implementação da classe. Em `String`, a intenção usual é `s1.equals(s2)`. Ao usar objetos próprios em `HashSet` ou como chaves de `HashMap`, `equals` e `hashCode` devem ser coerentes.

1.7 Passagem de parâmetros em Java

Java sempre passa argumentos **por valor**. Em tipos primitivos, copia o valor. Em objetos e vetores, copia o valor da referência: o método pode alterar o objeto apontado, mas trocar sua cópia local da referência não redireciona a variável do chamador.

O objeto muda; a variável do chamador, não

```
void alterar(Cor c) {
    c.red = 255; // altera o mesmo objeto
    c = new Cor(); // só troca a referência local
}
```

1.8 Modificadores mais relevantes

Modificador	Efeito operacional
private	acessível apenas na própria classe
sem modificador	acessível no mesmo pacote
protected	mesmo pacote e subclasses
public	acessível de qualquer lugar
static	membro da classe, não de cada objeto
final campo	recebe valor uma vez; constante quando também <code>static</code>
final método/classe	não pode ser sobrescrito / não pode ter subclasses
abstract	classe incompleta ou método sem implementação
synchronized	exclusão mútua no monitor correspondente

Atenção - campo escondido não é método sobrescrito

Se a subclasse declara um campo com o mesmo nome, os dois campos existem. `super.campo` acessa o herdado. Despacho dinâmico vale para métodos de instância sobrescritos, não para campos.

2 Relações entre classes, abstração e polimorfismo

Procure aqui: composição; herança; super; sobrescrita; despacho dinâmico; polimorfismo; classe abstrata; interface

2.1 Composição: um objeto tem ou usa outro

Definição direta - composição

Composição é representar colaboração entre objetos por meio de campos que guardam referências. Use-a quando uma classe **tem**, **possui** ou **usa** outra; ela permite distribuir responsabilidades sem afirmar que os tipos são da mesma família.

Pedido composto por outros objetos

```
class Pedido {
    private Cliente cliente;
    private Restaurante restaurante;
    private List<ItemPedido> itens;
    private Entregador entregador;
}
```

Decisão rápida - composição x herança

Pedido tem Cliente; portanto, um campo é adequado. **Aluno é Pessoa**; portanto, herança pode ser adequada. Prefira composição quando a relação “é um” não for verdadeira ou quando só existe reutilização de código.

2.2 Herança: especializar um tipo existente

Definição direta - herança

Herança cria uma subclasse mais específica a partir de uma superclasse. A subclasse recebe os membros acessíveis, pode acrescentar estado e pode especializar métodos. Java permite estender diretamente apenas uma classe.

Pessoa e Aluno; uso de super

```
class Pessoa {
    private String nome;
    Pessoa(String nome) { this.nome = nome; }
    String getName() { return nome; }
    void imprimir() { System.out.println(nome); }
}
class Aluno extends Pessoa {
    private String ra;
    Aluno(String nome, String ra) {
        super(nome); // constrói a parte Pessoa
        this.ra = ra;
    }
    @Override void imprimir() {
        super.imprimir(); // reutiliza método herdado
        System.out.println("RA: " + ra);
    }
}
```

Atenção - teste da relação “é um”

Pergunte: todo objeto da subclasse pode ser usado onde se espera a superclasse? Um Aluno é uma Pessoa. Um Pedido não é um Cliente; ele tem um cliente.

2.3 Sobrescrita (override) e despacho dinâmico

Definição direta - sobrescrita

Sobrescrita ocorre quando a subclasse fornece nova implementação para um método de instância herdado com assinatura compatível. `@Override` faz o compilador conferir a intenção. A versão executada depende da classe real do objeto.

Qual método executa?

```
class A {
    void m1() { System.out.println("A.m1"); m2(); }
    void m2() { System.out.println("A.m2"); }
}
class B extends A {
    @Override void m2() { System.out.println("B.m2"); }
}
A x = new B();
x.m1();
// A.m1
// B.m2 <- despacho pela classe real B
```

	Sobrecarga	Sobrescrita
Onde	mesma classe ou herança	exige herança
Parâmetros	diferentes	mesmos
Escolha	compilação	execução
Retorno	não distingue sozinho	igual ou covariante
Anotação	nenhuma	<code>@Override</code> recomendada

Erro frequente - limites da sobrescrita

A subclasse não pode reduzir a visibilidade. Construtores não são sobrescritos. Métodos `final` não podem ser sobrescritos; métodos `private` não são herdados de forma sobrescrevível; métodos `static` são ocultados, não despachados dinamicamente.

2.4 Polimorfismo: um contrato, várias implementações

Definição direta - polimorfismo

Polimorfismo permite tratar objetos de classes diferentes por um tipo comum, normalmente uma superclasse ou interface. O chamador usa o contrato comum; cada objeto executa sua própria implementação sobrescrita.

Coleção polimórfica

```
List<FormaGeometrica> formas = new ArrayList<FormaGeometrica>();
formas.add(new Circulo(2));
formas.add(new Retangulo(3, 4));
for (FormaGeometrica f : formas)
    System.out.println(f.calcularArea());
```

A referência `f` só permite chamar membros declarados em `FormaGeometrica`. Para acessar algo específico, confirme o tipo e faça cast apenas quando necessário:

Cast protegido

```
if (usuario instanceof Entregador) {
    Entregador e = (Entregador) usuario;
    e.setDisponivel(true);
}
```

2.5 Classe abstrata: base incompleta com estado comum

Definição direta - classe abstrata

Uma classe abstrata representa uma base que não deve ser instanciada. Pode ter campos, construtores, métodos concretos e métodos abstratos. Uma subclasse concreta precisa implementar os métodos abstratos herdados.

Base abstrata com operação obrigatória

```
abstract class Usuario {
    private String nome;
    Usuario(String nome) { this.nome = nome; }
    String getName() { return nome; }
    public abstract String visualizarPerfil();
}
class Cliente extends Usuario {
    Cliente(String nome) { super(nome); }
    @Override public String visualizarPerfil() {
        return "Cliente: " + getName();
    }
}
```

2.6 Interface: contrato entre tipos diferentes

Definição direta - interface

Uma interface declara quais operações um objeto oferece, sem exigir uma única hierarquia de classes. A classe usa `implements` e fornece os métodos obrigatórios. Isso permite polimorfismo entre classes não relacionadas e múltiplas capacidades na mesma classe.

MostraDados: contrato com sobrecarga

```
interface MostraDados {
    void mostra(int valor);
    void mostra(float valor);
}
class Mostra implements MostraDados {
    @Override public void mostra(int valor) {
        System.out.println("int: " + valor);
    }
    @Override public void mostra(float valor) {
        System.out.println("float: " + valor);
    }
}
static void mostraValores(MostraDados m) {
    m.mostra(4.56f);
    m.mostra(123);
}
```

O parâmetro é MostraDados, não Mostra; qualquer classe que cumpra o contrato pode ser usada sem alterar mostraValores.

Atenção - o que é implícito no Java 8

Campos de interface são public static final; métodos abstratos são public abstract. Métodos default e static podem ter implementação. Ao implementar um método abstrato, ele deve ser public. Uma interface pode estender várias interfaces; uma classe pode implementar várias interfaces.

2.7 Classe abstrata x interface

Pergunta	Classe abstrata	Interface
Estado de instância?	sim	não; apenas constantes
Construtor?	sim	não
Implementação?	métodos concretos e abstratos	abstratos, default e static
Quantas usar?	estende uma classe	implementa várias
Melhor uso	família com base comum	capacidade ou contrato comum
Palavra-chave	extends	implements

Decisão rápida - escolha rápida

Use **interface** quando classes possivelmente não relacionadas precisam oferecer a mesma operação: FormaGeometrica, Runnable, ActionListener. Use **classe abstrata** quando também existe estado e código comum: Usuario com nome, id e e-mail.

2.8 Exemplo central: FormaGeometrica

Contrato e duas implementações

```
interface FormaGeometrica {
    double PI = Math.PI;
    double calcularArea();
    double calcularPerimetro();
}
class Circulo implements FormaGeometrica {
    private double raio;
    Circulo(double raio) { this.raio = raio; }
    @Override public double calcularArea() {
        return PI * raio * raio;
    }
    @Override public double calcularPerimetro() {
        return 2 * PI * raio;
    }
}
class Retangulo implements FormaGeometrica {
    private double largura, altura;
    Retangulo(double l, double a) { largura = l; altura = a; }
    @Override public double calcularArea() {
        return largura * altura;
    }
    @Override public double calcularPerimetro() {
        return 2 * (largura + altura);
    }
}
```

Erro frequente - interfaces

- escrever extends Interface na classe em vez de implements Interface;
- esquecer public nos métodos implementados;
- tentar new FormaGeometrica();
- implementar apenas parte do contrato em classe concreta;
- declarar tudo com o tipo concreto e perder flexibilidade.

3 Exceções: detectar, lançar, propagar e recuperar

Procure aqui: exception; try; catch; finally; checked; unchecked; throw; throws; NumberFormatException; recursos

3.1 O que é uma exceção e como o fluxo muda

Definição direta - exceção

Uma exceção é um objeto que representa uma condição anormal ocorrida durante a execução. No ponto do erro, o fluxo normal é interrompido e o runtime procura, na pilha de chamadas, um catch compatível. Se ninguém tratar, o programa termina com o rastreamento da pilha.

Tratamento no método chamador

```
static int dividir(int a, int b) {
    return a / b; // pode lançar ArithmeticException
}
try {
    System.out.println(dividir(10, 0));
} catch (ArithmeticException e) {
    System.out.println("Divisão por zero");
}
System.out.println("Programa continua");
```

3.2 try, catch e finally

try delimita o trecho observado; catch trata um tipo; finally executa depois do try/catch mesmo quando ocorreu exceção, salvo encerramento abrupto da JVM.

Capturas específicas antes das gerais

```
try (FileInputStream in = new FileInputStream(nome)) {
    // leitura...
} catch (FileNotFoundException e) {
    System.out.println("Arquivo não encontrado");
} catch (IOException e) {
    System.out.println("Erro de entrada/saída");
} finally {
    System.out.println("Fim da tentativa");
}
```

Erro frequente - ordem dos catches

Capture primeiro a subclasse específica. FileNotFoundException deve vir antes de IOException; a classe geral primeiro tornaria o segundo catch inalcançável.

3.3 Checked x unchecked

	Checked	Unchecked
Família	subclasses de Exception, exceto RuntimeException	RuntimeException e subclasses
Compilador	exige capturar ou declarar	nada obrigatório
Exemplos	IOException; exceção própria estendendo Exception	IllegalArgumentException; NumberFormatException; Índice inválido
Uso típico	falha externa recuperável	argumento, uso ou estado inválido

Atenção - regra de compilação

Se um método pode produzir uma checked exception, ele deve tratá-la com try-catch ou declarar throws para transferir a responsabilidade ao chamador.

3.4 throw X throws

- throw new Excecao(...) cria e lança a instância naquele ponto;
- throws Excecao aparece na assinatura e declara que o método pode propagá-la.

Lançamento e propagação

```
static void validarSaldo(double saldo, double saque)
    throws SaldoInsuficienteException {
    if (saque > saldo)
        throw new SaldoInsuficienteException(saldo, saque);
}
```

3.5 Exceção personalizada

Uma exceção própria torna explícito um problema do domínio. Estenda Exception para checked ou RuntimeException para unchecked.

Saldo insuficiente como checked exception

```
class SaldoInsuficienteException extends Exception {
    SaldoInsuficienteException(double saldo, double saque) {
        super("Saldo " + saldo + ", saque pedido " + saque);
    }
}
class ContaBancaria {
    private double saldo;
    void sacar(double valor)
        throws SaldoInsuficienteException {
        if (valor <= 0)
            throw new IllegalArgumentException("Valor inválido");
        if (valor > saldo)
            throw new SaldoInsuficienteException(saldo, valor);
        saldo -= valor;
    }
}
```

3.6 Entrada inválida sem encerrar o programa

Leia a linha completa, tente converter e repita no catch. Isso evita que um token inválido permaneça no buffer do Scanner.

Função reutilizável de leitura

```
static int lerInt(Scanner sc, String mensagem) {
    while (true) {
        System.out.print(mensagem);
        try {
            return Integer.parseInt(sc.nextLine());
        } catch (NumberFormatException e) {
            System.out.println("Digite um número inteiro.");
        }
    }
}
```

Ao usar nextInt(), o token inválido não é consumido; no catch, chame nextLine() antes da nova tentativa. Misturar nextInt() e nextLine() também exige consumir a quebra de linha pendente.

3.7 Recursos e fechamento

Streams, scanners de arquivo e sockets devem ser fechados. try-with-resources fecha automaticamente tudo que implementa AutoCloseable, inclusive quando há exceção.

Fechamento automático

```
try (Scanner in = new Scanner(new FileInputStream(nome))) {
    while (in.hasNextLine())
        System.out.println(in.nextLine());
} catch (IOException e) {
    System.out.println("Falha ao ler: " + e.getMessage());
}
```

Erro frequente - erros que escondem o problema

- catch vazio ou genérico demais;
- lançar exceção depois de alterar parte do estado;
- usar exceção para um caso normal que poderia ser testado;
- declarar throws Exception em tudo sem decidir onde recuperar;
- esquecer fechamento quando não usa try-with-resources.

4 Collections, generics, iteração e ordenação

Procure aqui: List; ArrayList; Vector; LinkedList; Set; TreeSet; Map; TreeMap; Queue; Stack; Iterator; Enumeration; Comparable; Comparator

4.1 O que é uma coleção e como escolher

Definição direta - Collection e generics

Uma coleção agrupa referências a objetos em uma estrutura pronta para crescer, percorrer, buscar ou ordenar. Generics, como List<Aluno>, declaram o tipo aceito, impedem misturas indevidas e evitam casts inseguros.

Escolha pela propriedade que a questão precisa preservar, não pelo nome mais familiar.

Estrutura	Propriedade principal	Implementações úteis
List<T> Set<T>	ordem, duplicados e índice não aceita duplicados	ArrayList, LinkedList, Vector HashSet, LinkedHashSet, TreeSet
Queue<T> Map<K, V>	fila, normalmente FIFO chave associada a valor	LinkedList HashMap, TreeMap, Hashtable

Atenção - Map não é Collection

Map pertence ao framework, mas não estende Collection. Não aceita chaves duplicadas: um novo put para a mesma chave substitui o valor anterior.

4.2 List: ArrayList, Vector e LinkedList

Operações essenciais de List

```
List<String> nomes = new ArrayList<String>();
nomes.add("Maria");
nomes.add(0, "Ana");
String primeiro = nomes.get(0);
nomes.set(0, "Beatriz");
nomes.remove(1); // por índice
nomes.remove("Maria"); // por objeto
int n = nomes.size();
boolean existe = nomes.contains("Ana");
```

ArrayList é a escolha comum para lista com índice. Vector também cresce e mantém índice, mas seus métodos são sincronizados; use-o quando o exercício ou uma API exigir, como na lista de Collections. LinkedList é útil quando predominam operações nas extremidades e também implementa Queue.

Leitura e ordenação com Vector

```
Vector<String> nomes = new Vector<String>();
for (int i = 0; i < n; i++)
    nomes.add(sc.nextLine());
Collections.sort(nomes);
for (String nome : nomes)
    System.out.println(nome);
```

4.3 Set: unicidade e ordem

Definição direta - Set

Set representa conjunto: não admite dois elementos considerados iguais. HashSet não garante ordem; LinkedHashSet preserva inserção; TreeSet mantém ordenação pela ordem natural ou por Comparator.

TreeSet ordena e elimina repetidos

```
Set<String> nomes = new TreeSet<String>();
nomes.add("Maria");
nomes.add("João");
nomes.add("Maria"); // não altera o conjunto
for (String nome : nomes)
    System.out.println(nome);
```

Para classes próprias em estruturas hash, equals e hashCode precisam ser coerentes. Em TreeSet, compareTo ou compare define a ordem e também a equivalência prática: retorno zero significa “mesmo elemento” para o conjunto.

Erro frequente - comparador que perde elementos

Comparar Strings apenas por s1.length() - s2.length() faz palavras de mesmo tamanho retornarem zero. Desempate: return c != 0 ? c : s1.compareTo(s2);. Para inteiros gerais, prefira Integer.compare para evitar overflow.

4.4 Map: busca por chave e contagem

Definição direta - Map

Use Map quando há uma chave natural: RA para aluno, id para pedido ou nome da espécie para frequência. HashMap não ordena; TreeMap percorre as chaves ordenadas.

Contagem de ocorrências

```
Map<String, Integer> freq = new TreeMap<String, Integer>();
while (haEntrada) {
    String especie = lerLinha();
    freq.put(especie, freq.getDefault(especie, 0) + 1);
    total++;
}
for (Map.Entry<String, Integer> e : freq.entrySet()) {
    double p = 100.0 * e.getValue() / total;
    System.out.printf("%s %.4f%n", e.getKey(), p);
}
```

4.5 Fila e pilha

Fila segue FIFO; pilha segue LIFO.

Queue com LinkedList e Stack

```
Queue<Integer> fila = new LinkedList<Integer>();
fila.add(12);
fila.add(34);
while (!fila.isEmpty())
    System.out.println(fila.poll()); // retira o primeiro

Stack<String> pilha = new Stack<String>();
pilha.push("A");
pilha.push("B");
while (!pilha.empty())
    System.out.println(pilha.pop()); // retira o último
```

4.6 Percorrer: for-each, Iterator e Enumeration

O for-each é ideal para leitura simples. Iterator expõe hasNext, next e remoção segura. Enumeration é a forma antiga, ainda encontrada em Vector, com hasMoreElements e nextElement.

Três formas de percorrer

```
for (Aluno a : alunos) System.out.println(a);

Iterator<Aluno> it = alunos.iterator();
while (it.hasNext()) System.out.println(it.next());

Enumeration<String> e = vector.elements();
while (e.hasMoreElements())
    System.out.println(e.nextElement());
```

4.7 Remoção durante a iteração

Erro frequente - alteração estrutural por outro caminho

for (Aluno a : alunos) if (...) alunos.remove(a); pode lançar ConcurrentModificationException: o for-each usa um iterador internamente, mas a lista é modificada diretamente.

Iterator controla a remoção

```
Iterator<Aluno> it = alunos.iterator();
while (it.hasNext()) {
    Aluno a = it.next();
    if (a.getMedia() < 5.0)
        it.remove();
}
// Java 8, forma concisa:
// alunos.removeIf(a -> a.getMedia() < 5.0);
```

Também é possível percorrer índices do fim para o início: remover uma posição alta não desloca índices ainda não visitados.

4.8 Ordem natural e Comparador

Definição direta - duas formas de ordenar

Comparable<T> define a ordem natural dentro da própria classe.
Comparator<T> define um critério externo, útil para ordenar os mesmos objetos por nome, RA ou média sem alterar a classe.

Comparable: ordem natural pela RA

```
class Aluno implements Comparable<Aluno> {
    private String ra;
    @Override public int compareTo(Aluno outro) {
        return ra.compareTo(outro.ra);
    }
}
Collections.sort(alunos);
```

Comparator: média decrescente

```
Comparator<Aluno> porMedia = new Comparator<Aluno>() {
    @Override public int compare(Aluno a, Aluno b) {
        return Double.compare(b.getMedia(), a.getMedia());
    }
};
Collections.sort(alunos, porMedia);
```

4.9 Algoritmos da classe Collections

Método	Efeito
sort(lista)	ordem natural
sort(lista, comp)	ordem do comparador
reverse(lista)	inverte ordem atual
shuffle(lista)	embaralha
swap(lista,i,j)	troca posições
binarySearch(lista,x)	busca binária; mesma ordem da lista
min/max(colecao)	menor/maior elemento
synchronizedList(...)	wrapper sincronizado

Erro frequente - pegadinhas de Collections

- raw type, como List l, perde segurança e exige cast;
- Arrays.asList(vetor) tem tamanho fixo: add/remove falham;
- TreeMap ordena por chave, não por valor;
- get de chave ausente retorna null; cuidado com unboxing;
- Set não tem índice e pode descartar elemento equivalente;
- alterar o campo usado na ordenação pode quebrar a coerência de uma estrutura ordenada.

5 Swing, layouts, eventos e listeners

Procure aqui: JFrame; JButton; JTextField; JTextArea; JScrollPane; BorderLayout; GridLayout; FlowLayout; pack; ActionListener; Adapter; getSource

5.1 Estrutura mínima de uma janela Swing

Definição direta - GUI Swing

Swing fornece componentes gráficos iniciados por J. Um JFrame é um container de topo; componentes entram em containers e um gerenciador de layout decide tamanho e posição.

Janela Swing mínima

```
class Janela extends JFrame {
    Janela() {
        super("Título");
        add(new JButton("OK"));
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        pack();
        setVisible(true);
    }
    public static void main(String[] args) {
        SwingUtilities.invokeLater(new Runnable() {
            public void run() { new Janela(); }
        });
    }
}
```

Componente	Uso típico
JLabel	texto fixo ou resultado
JTextField	uma linha de entrada
JTextArea	várias linhas; normalmente em JScrollPane
JButton	ação clicável
JCheckBox	opções independentes
JRadioButton	opção exclusiva com ButtonGroup
JComboBox	lista suspensa
JProgressBar	progresso numérico
JOptionPane	diálogos de entrada, mensagem e confirmação

5.2 Layouts e painéis aninhados

Definição direta - gerenciador de layout

O layout calcula a posição e o tamanho dos componentes. Evite coordenadas fixas. Para composições maiores, crie painéis, dê a cada painel seu layout e combine-os no frame.

Layout	Regra
BorderLayout	cinco regiões: NORTH, SOUTH, EAST, WEST, CENTER; padrão do content pane de JFrame
GridLayout(r,c)	grade regular; células do mesmo tamanho
FlowLayout	sequência no tamanho preferido; padrão de JPanel

Erro frequente - BorderLayout substitui por região

Adicionar dois componentes diretamente à mesma região não os empilha: o último ocupa a região. Coloque-os em um JPanel e adicione o painel ao BorderLayout.

Editor: botões laterais + área rolável

```
JPanel botoes = new JPanel(new GridLayout(4, 1));
botoes.add(new JButton("Abrir"));
botoes.add(new JButton("Salvar"));
botoes.add(new JButton("Salvar Como"));
botoes.add(new JButton("Fechar"));
JTextArea texto = new JTextArea(10, 40);
add(botoes, BorderLayout.WEST);
add(new JScrollPane(texto), BorderLayout.CENTER);
```

JScrollPane cria barras quando o conteúdo excede a área visível. Adicione o scroll pane, não o JTextArea separadamente no mesmo lugar.

Calculadora: largura do botão

```
JPanel linha = new JPanel(new GridLayout(1, 5));
// campo, combo, campo, "=", resultado
add(linha, BorderLayout.CENTER);
add(new JButton("Calcular"), BorderLayout.SOUTH);
// acima: ocupa a largura da região

JPanel sul = new JPanel(); // FlowLayout
sul.add(new JButton("Calcular"));
add(sul, BorderLayout.SOUTH);
// acima: conserva tamanho preferido
```

5.3 pack(), fechamento e AWT x Swing

pack() dimensiona a janela pelos tamanhos preferidos dos componen- tes e layouts. setSize impõe tamanho externo. Defina também setDefaultCloseOperation e setVisible(true).

AWT usa componentes nativos como Frame e Button; Swing desenha componentes como JFrame e JButton em Java e oferece mais recursos. Classes de layout e even- tos de java.awt continuam sendo usadas com Swing. Não misture componentes AWT e Swing sem necessidade.

Erro frequente - erros visuais frequentes

- esquecer pack() OU setSize;
- esquecer visibilidade ou operação de fechamento;
- adicionar a mesma instância a dois containers;
- usar um único GridLayout quando painéis aninhados são necessários;
- adicionar JTextArea e JScrollPane ao mesmo lugar.

5.4 O que é um listener e os três passos obrigatórios

Definição direta - listener

Um listener é um objeto registrado para receber eventos. O componente é a fonte; quando ocorre uma ação, ele cria um objeto de evento e chama o método do listener registrado.

1. declarar classe que implementa a interface listener;
 2. implementar o método exigido;
 3. registrar a instância com add...Listener.

ActionListener completo

```
class Janela extends JFrame implements ActionListener {
    private JButton abrir = new JButton("Abrir");
    Janela() {
        add(abrir);
        abrir.addActionListener(this); // registro
    }
    @Override public void actionPerformed(ActionEvent e) {
        if (e.getSource() == abrir) {
            System.out.println("Abrir acionado");
        }
    }
}
```

5.5 Escolha do listener: evento semântico primeiro

Listener	Quando usar
ActionListener	botão, Enter em campo, item de menu
KeyListener	tecla pressionada/solta/digitada em baixo nível
MouseListener	clique, entrada/saída, pressionar/soltar mouse
MouseMotionListener	movimento e arraste
WindowListener	ciclo de vida da janela

Sempre que possível, prefira o evento semântico. Para botão, ActionListener ex- pressa a ação e também funciona com teclado; MouseListener capturaria apenas o mouse.

Assinaturas para reconhecer: KeyListener exige keyPressed, keyReleased e keyTyped; MouseListener exige mouseClicked, mousePressed, mouseReleased, mouseEntered e mouseExited. O registro usa addKeyListener OU addMouseListener.

5.6 Listener x Adapter e classes internas

Interfaces como MouseListener exigem vários métodos. Uma classe Adapter já fornece implementações vazias; sobrescreva apenas o evento necessário.

Adapter anônimo: só o clique

```
componente.addMouseListener(new MouseAdapter() {
    @Override public void mouseClicked(MouseEvent e) {
        System.out.println("Clique em " + e.getPoint());
    }
});
```

Como Java não permite estender duas classes, uma janela que já estende JFrame não pode também estender MouseAdapter. Uma classe interna ou anônima resolve o conflito.

5.7 Um listener para vários componentes

Registre this nos botões e diferencie a origem com e.getSource() ou comando de ação.

Contador com um ActionListener

```
public void actionPerformed(ActionEvent e) {
    if (e.getSource() == mais) {
        valor++;
        ultima.setText("Última: +1");
    } else if (e.getSource() == menos) {
        if (valor > 0) valor--;
        ultima.setText("Última: -1");
    } else if (e.getSource() == zerar) {
        valor = 0;
        ultima.setText("Última: zerar");
    }
    numero.setText(String.valueOf(valor));
}
```

A regra “não ficar negativo” pode estar no listener em exercício pequeno, mas em aplicação maior deve pertencer a um objeto-modelo chamado pela janela.

5.8 Como analisar um programa de GUI

1. identifique o layout do frame;
 2. liste a região de cada componente ou painel;
 3. abra cada painel e analise seu próprio layout;
 4. considere pack() e tamanhos preferidos;
 5. confira JScrollPane, ButtonGroup e listeners;
 6. só depois desenhe a janela resultante.

Ex.: GridLayout(1,4) em NORTH forma uma linha de quatro botões; JScrollPane em CENTER ocupa o restante.

6 Threads, sincronização e Swing

Procure aqui: Thread; Runnable; run; start; sleep; join; condição de corrida; synchronized; wait; notifyAll; EDT; invokeLater, Timer

6.1 O que é uma thread e duas formas de definir a tarefa

Definição direta - thread

Uma thread é um fluxo sequencial de execução dentro do programa. Vá- rias threads podem progredir concorrentemente; cada tarefa concorrente é escrita em run().

1. Herdar de Thread

```
class MinhaThread extends Thread {
    @Override public void run() {
        // tarefa concorrente
    }
}
new MinhaThread().start();
```

2. Implementar Runnable

```
class MinhaTarefa implements Runnable {
    @Override public void run() {
        // tarefa concorrente
    }
}
Thread t = new Thread(new MinhaTarefa());
t.start();
```

Prefira Runnable quando a classe já precisa estender outra, como JFrame, ou quando deseja separar a tarefa do mecanismo de execução.

Erro frequente - start não é run

start() solicita à JVM uma nova linha de execução, que chamará run(). Chamar run() diretamente é uma chamada normal e bloqueia a thread atual até terminar.

6.2 sleep e ordem de execução

Thread.sleep(ms) pausa a thread atual e pode lançar InterruptedException. Não libera monitores já adquiridos e não garante que uma thread específica execute em seguida.

Pausa com interrupção preservada

```
try {
    Thread.sleep(500);
} catch (InterruptedException e) {
    Thread.currentThread().interrupt();
}
```

Saídas de threads podem variar entre execuções. sleep muda temporização, não estabelece ordem nem substitui sincronização.

6.3 Esperar conclusão com join

Definição direta - join

t.join() suspende a thread atual até t terminar. Use antes de imprimir, combinar ou validar resultados produzidos em paralelo.

Iniciar todas, depois esperar todas

```
for (Thread t : threads) t.start();
for (Thread t : threads) {
    try {
        t.join();
    } catch (InterruptedException e) {
        Thread.currentThread().interrupt();
        return;
    }
}
System.out.println("Resultados completos");
```

Atenção - preservar paralelismo

Não faça start(); join(); para cada thread no mesmo laço: isso espera a primeira antes de iniciar a segunda. Primeiro inicie todas; depois espere todas.

6.4 Soma de matrizes por regiões exclusivas

Divida a matriz de resultado em regiões sem sobreposição. Cada worker recebe limites com fim exclusivo e escreve apenas em sua parte; assim, as células não precisam de lock. O join garante que a impressão ocorra depois do preenchimento.

Worker por intervalo de linhas

```
class SomaLinhas implements Runnable {
    private int[][] a, b, c;
    private int inicio, fim; // fim exclusivo
    public void run() {
        for (int i = inicio; i < fim; i++)
            for (int j = 0; j < a[i].length; j++)
                c[i][j] = a[i][j] + b[i][j];
    }
}
```

Para quatro quadrantes, forneça também limites de colunas. Se N for ímpar, escolha o meio de modo que as quatro regiões cubram todos os índices uma única vez; a solução completa está em p. 21.

6.5 Condição de corrida e operação atômica

Definição direta - condição de corrida

Condição de corrida ocorre quando o resultado depende da intercalação imprevisível de acessos ao mesmo estado. c++ não é atômico: lê, calcula e grava; outra thread pode intercalar esses passos e perder uma atualização.

Métodos que protegem o mesmo estado

```
class Contador {
    private int c;
    public synchronized void incrementar() { c++; }
    public synchronized void decrementar() { c--; }
    public synchronized int valor() { return c; }
}
```

Um método de instância synchronized usa como monitor o próprio objeto. Duas threads não executam simultaneamente trechos sincronizados sobre a mesma instância; instâncias diferentes possuem locks diferentes.

6.6 Blocos sincronizados e granularidade

Use blocos quando apenas parte do método precisa de exclusão mútua ou quando dados independentes podem usar locks diferentes.

Locks independentes

```
private final Object lock1 = new Object();
private final Object lock2 = new Object();
private long c1, c2;
void inc1() { synchronized (lock1) { c1++; } }
void inc2() { synchronized (lock2) { c2++; } }
```

Mantenha a região crítica curta, mas inclua toda a sequência que precisa ser indivisível. Sincronizar apenas a leitura e não a escrita não protege a regra completa.

6.7 Produtor-consumidor: wait e notifyAll

Um laço vazio como while (disponivel); ocupa CPU. Se executado mantendo o monitor, impede a outra thread de mudar a condição. wait() suspende a thread e libera o monitor.

Depósito de uma posição

```
class Deposito {
    private int conteudo;
    private boolean disponivel;

    synchronized void put(int valor)
        throws InterruptedException {
        while (!disponivel) wait();
        conteudo = valor;
        disponivel = true;
        notifyAll();
    }

    synchronized int get()
        throws InterruptedException {
        while (!disponivel) wait();
        int valor = conteudo;
        disponivel = false;
        notifyAll();
        return valor;
    }
}
```

Use while, não if: ao acordar, a thread precisa testar novamente a condição. wait, notify e notifyAll só podem ser chamados quando a thread possui o monitor correspondente.

6.8 Regra da thread única do Swing (EDT)

Definição direta - EDT

A Event Dispatch Thread processa eventos e atualizações visuais do Swing. Listeners já executam nela. Tarefa demorada na EDT impede pintura e novos eventos; a janela parece congelada.

Situação	Regra
Criar e mostrar GUI	submeter com invokeLater
actionPerformed	já está na EDT; atualização curta pode ser direta
Tarefa demorada	executar em worker fora da EDT
Atualização vinda da worker	publicar trecho curto na EDT
repaint()	pode ser solicitado de outra thread
javax.swing.Timer	listener roda na EDT

Worker atualizando barra de progresso

```
public void run() {
    for (int i = 0; i <= 10; i++) {
        final int valor = i;
        SwingUtilities.invokeLater(new Runnable() {
            public void run() { barra.setValue(valor); }
        });
        try { Thread.sleep(500); }
        catch (InterruptedException e) { return; }
    }
}
```

Erro frequente - por que não dormir no listener

actionPerformed roda na EDT. Um laço com sleep dentro dele impede repintura e eventos. O listener deve iniciar a worker, desabilitar o botão e retornar rapidamente.

6.9 invokeLater X invokeAndWait

	invokeLater	invokeAndWait
Retorno	imediatamente	após executar na EDT
Uso	atualização assíncrona	precisa continuar com resultado já aplicado
Na própria EDT	permitido, agenda depois	proibido; causaria bloqueio
Exceções	simples	interrupção e InvocationTargetException

6.10 Timer Swing

javax.swing.Timer é adequado para ações pequenas e periódicas ligadas à interface. Seu listener roda na EDT e pode alterar componentes diretamente, mas não deve realizar trabalho demorado.

Atualização periódica na EDT

```
Timer timer = new Timer(1000, new ActionListener() {
    @Override public void actionPerformed(ActionEvent e) {
        barra.setValue(cont++);
    }
});
timer.start();
```

Erro frequente - threads

- chamar run em vez de start;
- usar resultado antes de join;
- dividir intervalos com lacunas ou sobreposição;
- acreditar que sleep sincroniza;
- sincronizar instâncias diferentes sem perceber;
- usar wait fora do monitor;
- alterar Swing diretamente da worker;
- executar rede, arquivo ou cálculo longo na EDT.

7 Streams, arquivos e sockets

Procure aqui: InputStream; OutputStream; Reader; Writer; buffer; Scanner; serialização; File; Socket; ServerSocket; protocolo

7.1 O que é um stream e como escolher

Definição direta - stream

Um stream é uma sequência de dados entre o programa e uma fonte ou destino. A família escolhida depende de o conteúdo ser byte, caractere, texto formatado, primitivo binário ou objeto.

Família	Uso
InputStream/OutputStream	bytes brutos; arquivos binários e base dos sockets
Reader/Writer	caracteres e conversão de codificação
BufferedReader/Writer	buffer e operações convenientes
Scanner	separa texto e converte tipos
PrintWriter/PrintStream	texto e formatação
DataInput/OutputStream	primitivos e String em binário
ObjectInput/OutputStream	objetos serializáveis
RandomAccessFile	leitura/escrita com ponteiro e seek

7.2 Bytes, caracteres e buffer

Use bytes para conteúdo binário ou quando a API fornece streams de bytes. Use Reader/Writer para texto. InputStreamReader e OutputStreamWriter fazem a ponte. Um buffer reduz chamadas ao sistema.

Leitura de texto com buffer

```
try (BufferedReader in = new BufferedReader(
    new FileReader("arquivo.txt"))) {
    String linha;
    while ((linha = in.readLine()) != null)
        System.out.println(linha);
}
```

Escrita de texto formatado

```
try (PrintWriter out = new PrintWriter("saida.txt")) {
    out.printf("Total: %.2f%n", total);
}
```

7.3 Scanner, tokens e delimitadores

Scanner pode ler teclado, String ou arquivo. Consulte com hasNext... quando a entrada pode terminar ou conter tipos diferentes.

Leitura tipada de arquivo

```
try (Scanner sc = new Scanner(new File("dados.txt"))) {
    while (sc.hasNextLong()) {
        long valor = sc.nextLong();
    }
}
```

Misturar nextInt() e nextLine() exige consumir a quebra pendente. Para nomes com espaços, leia a linha completa e converta os números.

7.4 Objetos, serialização e a classe File

ObjectOutputStream e ObjectOutputStream gravam e leem grafos de objetos por writeObject e readObject. As classes devem implementar Serializable; a leitura devolve Object e exige cast compatível.

Fluxo de objetos

```
class Aluno implements Serializable { /* campos */ }
out.writeObject(aluno);
Aluno copia = (Aluno) in.readObject();
```

File representa um caminho, não o conteúdo nem a garantia de existência. Permite testar, listar, renomear e apagar; abrir dados ainda exige stream, reader ou scanner.

7.5 Sockets: papéis e conexão TCP

Definição direta - socket

Um socket é uma extremidade de comunicação bidirecional. O servidor escolhe uma porta e espera; o cliente conhece host/IP e porta e inicia a conexão. Depois, ambos trocam dados pelos streams do Socket.

Etapas	Servidor	Cliente
Criar	new ServerSocket(porta)	new Socket(host,porta)
Conectar	accept() bloqueia e devolve Socket	construtor conecta
Trocar	streams do Socket	streams do Socket
Finalizar	fechar socket e servidor	fechar socket

Servidor de uma conexão

```
try (ServerSocket servidor = new ServerSocket(5000);
    Socket cliente = servidor.accept();
    BufferedReader in = new BufferedReader(
        new InputStreamReader(cliente.getInputStream()));
    PrintWriter out = new PrintWriter(
        cliente.getOutputStream(), true) {
    String mensagem = in.readLine();
    out.println("Eco: " + mensagem);
}
```

Cliente correspondente

```
try (Socket socket = new Socket("localhost", 5000);
    BufferedReader in = new BufferedReader(
        new InputStreamReader(socket.getInputStream()));
    PrintWriter out = new PrintWriter(
        socket.getOutputStream(), true) {
    out.println("olá");
    System.out.println(in.readLine());
}
```

7.6 Protocolo: ordem e formato das mensagens

Definição direta - protocolo

Protocolo é o acordo entre cliente e servidor sobre quem envia primeiro, quais mensagens existem, como cada uma é formatada e quando a sessão termina. Sem esse acordo, código correto pode bloquear ou interpretar dados errados.

Se ambos chamarem readLine() primeiro, podem esperar para sempre. Em protocolo por linhas, toda mensagem precisa terminar com quebra; PrintWriter(..., true) faz auto-flush em println.

Decisão rápida - antes de analisar um socket

Desenhe duas colunas, servidor e cliente. Alinhe cada write/println ao read/readLine correspondente. Marque quem bloqueia em accept, quem envia primeiro, como o fim é sinalizado e o que ocorre quando readLine() retorna null.

Erro frequente - sockets

Portas diferentes; IP incorreto; falta de flush; leituras e escritas em ordens incompatíveis; rede na EDT; ignorar readLine()==null; não fechar recursos.

8 Exercícios de implementação resolvidos

Esta seção reúne os exercícios que exigem escrever, alterar ou estruturar código. Cada agrupamento identifica a lista do professor e cada item começa pelo contexto mínimo e pelos conceitos demonstrados. Programas completos permanecem em arquivos próprios; nos exercícios de modelagem ou análise, o núcleo da solução aparece diretamente.

8.1 Lista de revisão - exercícios 2 a 6

8.1.1 Exercício 2 - Produto e maior valor em estoque

Contexto e leitura do código

Modele produto com nome, preço e quantidade. A classe oferece construtor, getters/setters, reposição, venda, cálculo do valor total e toString. O programa lê três produtos e mostra o maior valor em estoque. Esta é deliberadamente a versão-base do exercício, ainda sem as invariantes acrescentadas no Exercício 3.
Conceitos: encapsulamento estrutural, construtor, vetor de objetos, leitura com Scanner e comparação de valores.

Revisao2Produto.java

```
import java.util.Scanner;

class Produto {
    private String nome;
    private double preco;
    private int quantidade;

    Produto(String nome, double preco, int quantidade) {
        this.nome = nome;
        this.preco = preco;
        this.quantidade = quantidade;
    }

    String getNome() { return nome; }
    double getPreco() { return preco; }
    int getQuantidade() { return quantidade; }

    void setNome(String nome) { this.nome = nome; }
    void setPreco(double preco) { this.preco = preco; }
    void setQuantidade(int quantidade) { this.quantidade = quantidade; }

    void reporEstoque(int quantidade) {
        this.quantidade += quantidade;
    }

    void vender(int quantidade) {
        this.quantidade -= quantidade;
    }

    double calcularValorEmEstoque() {
        return preco * quantidade;
    }

    @Override
    public String toString() {
        return nome + " | preço=" + preco + " | qtd=" + quantidade;
    }
}

public class Revisao2Produto {
    static Produto lerProduto(Scanner sc, int numero) {
        System.out.println("Produto " + numero);
        System.out.print("Nome: ");
        String nome = sc.nextLine();
        System.out.print("Preço: ");
        double preco = Double.parseDouble(sc.nextLine());
        System.out.print("Quantidade: ");
        int quantidade = Integer.parseInt(sc.nextLine());
        return new Produto(nome, preco, quantidade);
    }

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        Produto[] produtos = new Produto[3];

        for (int i = 0; i < produtos.length; i++)
            produtos[i] = lerProduto(sc, i + 1);

        Produto maior = produtos[0];
        for (int i = 1; i < produtos.length; i++) {
            if (produtos[i].calcularValorEmEstoque()
                > maior.calcularValorEmEstoque()) {
                maior = produtos[i];
            }
        }
        System.out.println("Maior valor em estoque: " + maior);
    }
}
```

8.1.2 Exercício 3 - Produto com invariantes e recuperação de entrada

Contexto e diferença em relação ao exercício anterior

A classe agora rejeita preço ou estoque negativos, venda acima do estoque e venda/reposição não positiva. Violações do domínio lançam IllegalArgumentException; texto em campo numérico é capturado e a leitura recomeça.
Conceitos: invariante, validação no ponto de alteração, exceção, repetição de leitura e estado sempre válido.

Revisao3ProdutoSeguro.java

```
import java.util.Scanner;

class ProdutoSeguro {
    private String nome;
    private double preco;
    private int quantidade;

    ProdutoSeguro(String nome, double preco, int quantidade) {
        this.nome = nome;
        setPreco(preco);
        setQuantidade(quantidade);
    }

    String getName() { return nome; }
    double getPreco() { return preco; }
    int getQuantidade() { return quantidade; }

    void setName(String nome) { this.nome = nome; }

    void setPreco(double preco) {
        if (preco < 0)
            throw new IllegalArgumentException("Preço negativo");
        this.preco = preco;
    }

    void setQuantidade(int quantidade) {
        if (quantidade < 0)
            throw new IllegalArgumentException("Estoque negativo");
        this.quantidade = quantidade;
    }

    void reporEstoque(int quantidade) {
        if (quantidade <= 0)
            throw new IllegalArgumentException(
                "Reposição deve ser positiva");
        this.quantidade += quantidade;
    }

    void vender(int quantidade) {
        if (quantidade <= 0)
            throw new IllegalArgumentException(
                "Venda deve ser positiva");
        if (quantidade > this.quantidade)
            throw new IllegalArgumentException("Estoque insuficiente");
        this.quantidade -= quantidade;
    }

    double calcularValorEmEstoque() {
        return preco * quantidade;
    }

    @Override
    public String toString() {
        return nome + " | preço=" + preco + " | qtd=" + quantidade;
    }
}

public class Revisao3ProdutoSeguro {
    static ProdutoSeguro lerProduto(Scanner sc, int numero) {
        while (true) {
            try {
                System.out.println("Produto " + numero);
                System.out.print("Nome: ");
                String nome = sc.nextLine();
                System.out.print("Preço: ");
                double preco = Double.parseDouble(sc.nextLine());
                System.out.print("Quantidade: ");
                int quantidade = Integer.parseInt(sc.nextLine());
                return new ProdutoSeguro(nome, preco, quantidade);
            } catch (NumberFormatException e) {
                System.out.println("Digite números válidos.\n");
            } catch (IllegalArgumentException e) {
                System.out.println(e.getMessage() + ". Tente novamente.\n");
            }
        }
    }

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        ProdutoSeguro[] produtos = new ProdutoSeguro[3];

        for (int i = 0; i < produtos.length; i++)
            produtos[i] = lerProduto(sc, i + 1);

        ProdutoSeguro maior = produtos[0];
        for (int i = 1; i < produtos.length; i++) {
            if (produtos[i].calcularValorEmEstoque()
                > maior.calcularValorEmEstoque()) {
                maior = produtos[i];
            }
        }
        System.out.println("Maior valor em estoque: " + maior);
    }
}
```


8.1.3 Exercício 4 - Cadastro de alunos com menu e RA único

Contexto

Mantenha alunos com nome, RA e média em ArrayList. O menu cadastra, lista, pesquisa, remove, calcula a média da turma e mostra aprovados. Um RA existente é recusado. Concentre a busca por RA em um método e reutilize-o para impedir duplicidade, pesquisar e remover.

Conceitos: coleção, busca por chave, remoção, agregação, filtro e menu de console.

Revisao4Alunos.java

```
import java.util.ArrayList;
import java.util.List;
import java.util.Scanner;

class Aluno {
    private final String nome;
    private final String ra;
    private final double media;

    Aluno(String nome, String ra, double media) {
        this.nome = nome;
        this.ra = ra;
        this.media = media;
    }

    String getRa() { return ra; }
    double getMedia() { return media; }

    @Override
    public String toString() {
        return nome + " | RA=" + ra + " | média=" + media;
    }
}

public class Revisao4Alunos {
    static Aluno buscarPorRa(List<Aluno> alunos, String ra) {
        for (Aluno a : alunos)
            if (a.getRa().equals(ra)) return a;
        return null;
    }

    static boolean cadastrar(List<Aluno> alunos, Aluno novo) {
        if (buscarPorRa(alunos, novo.getRa()) != null) return false;
        alunos.add(novo);
        return true;
    }

    static double mediaTurma(List<Aluno> alunos) {
        if (alunos.isEmpty()) return 0;
        double soma = 0;
        for (Aluno a : alunos) soma += a.getMedia();
        return soma / alunos.size();
    }

    static void listar(List<Aluno> alunos) {
        if (alunos.isEmpty()) {
            System.out.println("Nenhum aluno.");
        } else {
            for (Aluno a : alunos) System.out.println(a);
        }
    }

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        List<Aluno> alunos = new ArrayList<Aluno>();
        int opcao = -1;

        do {
            System.out.println("1-Cadastrar 2-Listar 3-Pesquisar 4-Remover");
            System.out.println("5-Média 6-Aprovados 0-Encerrar");
            try {
                opcao = Integer.parseInt(sc.nextLine());
                switch (opcao) {
                    case 1:
                        System.out.print("Nome: ");
                        String nome = sc.nextLine();
                        System.out.print("RA: ");
                        String ra = sc.nextLine();
                        if (buscarPorRa(alunos, ra) != null) {
                            System.out.println("RA já cadastrado.");
                            break;
                        }
                        System.out.print("Média: ");
                        double media = Double.parseDouble(sc.nextLine());
                        cadastrar(alunos, new Aluno(nome, ra, media));
                        break;
                    case 2:
                        listar(alunos);
                        break;
                    case 3:
                        System.out.print("RA: ");
                        Aluno achado = buscarPorRa(alunos, sc.nextLine());
                        System.out.println(achado == null
                            ? "Não encontrado" : achado);
                        break;
                    case 4:
```

```

        System.out.print("RA: ");
        Aluno removido = buscarPorRa(alunos, sc.nextLine());
        System.out.println(removido != null
            && alunos.remove(removido)
            ? "Removido" : "Não encontrado");
        break;
    case 5:
        System.out.printf("Média: %.2f%n",
            mediaTurma(alunos));
        break;
    case 6:
        for (Aluno a : alunos)
            if (a.getMedia() >= 5.0)
                System.out.println(a);
        break;
    case 0:
        break;
    default:
        System.out.println("Opção inválida.");
    }
} catch (NumberFormatException e) {
    System.out.println("Digite um número válido.");
    opcao = -1;
}
} while (opcao != 0);
}
}

```

8.1.4 Exercício 5 - Remoção segura de alunos reprovados

Por que o for-each falha

Remover diretamente da lista dentro do for-each altera a estrutura por um caminho diferente do iterador usado pelo laço e pode lançar `ConcurrentModificationException`. A solução principal usa `Iterator.remove`; a forma `removeIf` aparece como abreviação Java 8.

Revisao5Remocao.java

```

import java.util.Iterator;
import java.util.List;

class Aluno {
    private final double media;

    Aluno(double media) {
        this.media = media;
    }

    double getMedia() {
        return media;
    }
}

public class Revisao5Remocao {
    static void removerReprovados(List<Aluno> alunos) {
        Iterator<Aluno> it = alunos.iterator();
        while (it.hasNext()) {
            if (it.next().getMedia() < 5.0)
                it.remove();
        }

        // Forma concisa disponível no Java 8:
        // alunos.removeIf(a -> a.getMedia() < 5.0);
    }
}

```

8.1.5 Exercício 6 - Contador Swing

Contexto

A janela mostra valor atual, botões +1, -1 e Zerar e um rótulo da última operação. O valor não fica negativo. Um único `ActionListener` é registrado nos três botões e distingue a fonte por `getSource`.

Conceitos: estado da interface, registro de listener, atualização de rótulos e criação da GUI na EDT.

Revisao6Contador.java

```

import java.awt.BorderLayout;
import java.awt.Font;
import java.awt.event.ActionEvent;
import java.awt.event.ActionListener;
import javax.swing.*;

public class Revisao6Contador extends JFrame
    implements ActionListener {
    private int contador;
    private final JLabel valor = new JLabel(
        "0", SwingConstants.CENTER);
    private final JLabel ultima = new JLabel(
        "Última operação: nenhuma", SwingConstants.CENTER);
    private final JButton mais = new JButton("+1");
    private final JButton menos = new JButton("-1");
    private final JButton zerar = new JButton("Zerar");

    Revisao6Contador() {

```

```

        super("Contador");
        valor.setFont(new Font("SansSerif", Font.BOLD, 32));

        JPanel botoes = new JPanel();
        botoes.add(mais);
        botoes.add(menos);
        botoes.add(zerar);

        mais.addActionListener(this);
        menos.addActionListener(this);
        zerar.addActionListener(this);

        add(valor, BorderLayout.NORTH);
        add(botoes, BorderLayout.CENTER);
        add(ultima, BorderLayout.SOUTH);
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        pack();
        setLocationRelativeTo(null);
        setVisible(true);
    }

    @Override
    public void actionPerformed(ActionEvent e) {
        if (e.getSource() == mais) {
            alterar(contador + 1, "+1");
        } else if (e.getSource() == menos) {
            alterar(Math.max(0, contador - 1), "-1");
        } else if (e.getSource() == zerar) {
            alterar(0, "Zerar");
        }
    }

    private void alterar(int novoValor, String operacao) {
        contador = novoValor;
        valor.setText(String.valueOf(contador));
        ultima.setText("Última operação: " + operacao);
    }

    public static void main(String[] args) {
        SwingUtilities.invokeLater(new Runnable() {
            public void run() {
                new Revisao6Contador();
            }
        });
    }
}

```

8.2 Lista 3 - Sistema de Delivery

8.2.1 Exercício - modelagem e núcleo do pedido

Contexto

O exercício reúne as três relações principais: Cliente, Entregador e Restaurante são especializações de Usuario; Pedido é composto por participantes, itens e estado; SistemaDelivery mantém coleções e coordena operações.

Definição direta - enum

enum define um conjunto fechado de valores nomeados. É adequado para estados como pedido realizado, em preparo, em entrega ou entregue, pois impede Strings arbitrárias e facilita validação de transições.

Núcleo do Pedido

```

enum StatusPedido {
    REALIZADO, EM_PREPARO, EM_ENTREGA, ENTREGUE
}

class Pedido {
    private Cliente cliente;
    private Restaurante restaurante;
    private List<ItemPedido> itens;
    private Entregador entregador;
    private StatusPedido status = StatusPedido.REALIZADO;

    double calcularTotal() {
        double total = 0;
        for (ItemPedido item : itens) total += item.getValor();
        return total;
    }

    void atribuirEntregador(Entregador e) {
        if (!e.isDisponivel())
            throw new IllegalStateException("Indisponível");
        entregador = e;
        e.setDisponivel(false);
    }
}

```

Decisão rápida - ordem para modelos com várias classes

1) defina entidades e invariantes; 2) marque "é um", "tem um" e contratos; 3) crie uma classe gerenciadora para buscas e coordenação; 4) deixe o menu apenas converter opções em chamadas de métodos.

8.3 Lista 4 - Exceções - exercícios 1 e 2

8.3.1 Exercício 1 - Equação quadrática com exceções distintas

Contexto

O discriminante $\Delta = b^2 - 4ac$ define três casos. O exercício pede resolver apenas duas raízes reais distintas e lançar exceções diferentes nos demais.

Decisão pelo discriminante

```
if (a == 0)
    throw new IllegalArgumentException("Não é 2º grau");
double delta = b * b - 4 * a * c;
if (delta < 0) throw new SemRaizesReaisException();
if (delta == 0) throw new RaizUnicaException();
double x1 = (-b + Math.sqrt(delta)) / (2 * a);
double x2 = (-b - Math.sqrt(delta)) / (2 * a);
```

Com coeficientes inteiros, comparar `delta == 0` é direto. Se o discriminante resultar de cálculos em ponto flutuante, use tolerância para decidir se está aproximadamente em zero.

8.3.2 Exercício 2 - Vetor limitado e múltiplas exceções

Contexto

O exercício aceita inteiros até incluir o valor 0 em um vetor de 10 posições. Há erros independentes: texto não numérico e tentativa de gravar além do limite.

Recuperação dos dois casos

```
try {
    int i = 0, valor;
    do {
        valor = sc.nextInt();
        vet[i++] = valor;
    } while (valor != 0);
} catch (InputMismatchException e) {
    System.out.println("Entrada não numérica");
} catch (ArrayIndexOutOfBoundsException e) {
    System.out.println("Mais de 10 valores");
}
```

8.4 Lista 5 - Collections - exercício 3

8.4.1 Exercício 3 - Formar duplas com menor diferença

Contexto

Para formar duplas com menor soma de diferenças, ordene as habilidades e emparelhe valores vizinhos. No exercício 1092-B, a quantidade de habilidades é par; em outro enunciado, valide o tamanho antes de acessar `i + 1`.

Ordenar e emparelhar duas sequências

```
Collections.sort(habilidades);
int custo = 0;
for (int i = 0; i < habilidades.size(); i += 2)
    custo += habilidades.get(i + 1) - habilidades.get(i);
System.out.println(custo);
```

8.5 Lista 6 - GUI - exercícios 1 a 4

8.5.1 Exercício 1 - Editor: botões laterais e área rolável

Contexto

Quatro botões em coluna à esquerda e uma área de texto no centro. O painel lateral usa `GridLayout(4,1)`; o frame usa `BorderLayout`; a área fica em `JScrollPane`. O botão Fechar apenas demonstra um listener simples.

GUI1Editor.java

```
import java.awt.BorderLayout;
import java.awt.GridLayout;
import java.awt.event.ActionEvent;
import java.awt.event.ActionListener;
import javax.swing.*;

public class GUI1Editor extends JFrame {
    GUI1Editor() {
        super("Editor de Texto");

        JPanel botoes = new JPanel(new GridLayout(4, 1));
        JButton abrir = new JButton("Abrir");
        JButton salvar = new JButton("Salvar");
        JButton salvarComo = new JButton("Salvar Como");
        JButton fechar = new JButton("Fechar");
        botoes.add(abrir);
        botoes.add(salvar);
        botoes.add(salvarComo);
        botoes.add(fechar);

        JTextArea texto = new JTextArea(12, 40);
        fechar.addActionListener(new ActionListener() {
            public void actionPerformed(ActionEvent e) {
                dispose();
            }
        });
    }
}
```

```

        add(botoes, BorderLayout.WEST);
        add(new JScrollPane(texto), BorderLayout.CENTER);
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        pack();
        setLocationRelativeTo(null);
        setVisible(true);
    }

    public static void main(String[] args) {
        SwingUtilities.invokeLater(new Runnable() {
            public void run() {
                new GUI1Editor();
            }
        });
    }
}

```

8.5.2 Exercício 2 - Calculadora com quatro botões de operação

Contexto

Dois campos numéricos, quatro botões verticais, um rótulo “=” e o resultado. A classe implementa ActionListener, registra a mesma instância e converte a fonte em uma operação. Entrada inválida e divisão por zero são apresentadas por diálogo.

GUI2CalculadoraBotoes.java

```

import java.awt.FlowLayout;
import java.awt.GridLayout;
import java.awt.event.ActionEvent;
import java.awt.event.ActionListener;
import javax.swing.*;

public class GUI2CalculadoraBotoes extends JFrame
    implements ActionListener {
    private final JTextField a = new JTextField(8);
    private final JTextField b = new JTextField(8);
    private final JLabel resultado = new JLabel("0");
    private final JButton soma = new JButton("Soma");
    private final JButton subtrai = new JButton("Subtrai");
    private final JButton multiplica = new JButton("Multiplica");
    private final JButton divide = new JButton("Divide");

    GUI2CalculadoraBotoes() {
        super("Calculadora");

        JPanel operacoes = new JPanel(new GridLayout(4, 1));
        JButton[] botoes = { soma, subtrai, multiplica, divide };
        for (JButton botao : botoes) {
            botao.addActionListener(this);
            operacoes.add(botao);
        }

        JPanel linha = new JPanel(new FlowLayout());
        linha.add(a);
        linha.add(operacoes);
        linha.add(b);
        linha.add(new JLabel("="));
        linha.add(resultado);
        add(linha);

        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        pack();
        setLocationRelativeTo(null);
        setVisible(true);
    }

    @Override
    public void actionPerformed(ActionEvent e) {
        char op;
        if (e.getSource() == soma) op = '+';
        else if (e.getSource() == subtrai) op = '-';
        else if (e.getSource() == multiplica) op = '*';
        else op = '/';
        calcular(op);
    }

    private void calcular(char op) {
        try {
            double x = Double.parseDouble(a.getText());
            double y = Double.parseDouble(b.getText());
            if (op == '/' && y == 0)
                throw new ArithmeticException("Divisão por zero");

            double r;
            if (op == '+') r = x + y;
            else if (op == '-') r = x - y;
            else if (op == '*') r = x * y;
            else r = x / y;
            resultado.setText(String.valueOf(r));
        } catch (NumberFormatException e) {
            JOptionPane.showMessageDialog(this, "Entrada inválida");
        } catch (ArithmeticException e) {
            JOptionPane.showMessageDialog(this, e.getMessage());
        }
    }
}

```

```

    }

    public static void main(String[] args) {
        SwingUtilities.invokeLater(new Runnable() {
            public void run() {
                new GUI2CalculadoraBotoes();
            }
        });
    }
}

```

8.5.3 Exercícios 3 e 4 - JComboBox e largura do botão

Contexto

A linha superior reúne campo, operação, campo, “=” e resultado. Sem argumento, o botão entra diretamente em SOUTH e ocupa a largura; com o argumento compacto, ele fica em um JPanel com FlowLayout e conserva o tamanho preferido.

GUI34CalculadoraCombo.java

```

import java.awt.BorderLayout;
import java.awt.FlowLayout;
import java.awt.GridLayout;
import java.awt.event.ActionEvent;
import java.awt.event.ActionListener;
import javax.swing.*;

public class GUI34CalculadoraCombo extends JFrame
    implements ActionListener {
    private final JTextField a = new JTextField(8);
    private final JTextField b = new JTextField(8);
    private final JComboBox<String> operacao = new JComboBox<String>(
        new String[] { "Soma", "Subtrai", "Multiplica", "Divide" });
    private final JLabel resultado = new JLabel("0");
    private final JButton calcular = new JButton("Calcular");

    GUI34CalculadoraCombo(boolean botaoCompacto) {
        super("Calculadora");

        JPanel linha = new JPanel(new GridLayout(1, 5, 4, 0));
        linha.add(a);
        linha.add(operacao);
        linha.add(b);
        linha.add(new JLabel("="));
        linha.add(resultado);

        calcular.addActionListener(this);
        add(linha, BorderLayout.CENTER);
        if (botaoCompacto) {
            JPanel sul = new JPanel(new FlowLayout());
            sul.add(calcular);
            add(sul, BorderLayout.SOUTH);
        } else {
            add(calcular, BorderLayout.SOUTH);
        }

        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        pack();
        setLocationRelativeTo(null);
        setVisible(true);
    }

    @Override
    public void actionPerformed(ActionEvent e) {
        try {
            double x = Double.parseDouble(a.getText());
            double y = Double.parseDouble(b.getText());
            int op = operacao.getSelectedIndex();
            if (op == 3 && y == 0)
                throw new ArithmeticException("Divisão por zero");

            double r;
            if (op == 0) r = x + y;
            else if (op == 1) r = x - y;
            else if (op == 2) r = x * y;
            else r = x / y;
            resultado.setText(String.valueOf(r));
        } catch (NumberFormatException ex) {
            JOptionPane.showMessageDialog(this, "Entrada inválida");
        } catch (ArithmeticException ex) {
            JOptionPane.showMessageDialog(this, ex.getMessage());
        }
    }

    public static void main(String[] args) {
        final boolean compacto = args.length > 0
            && args[0].equals("compacto");
        SwingUtilities.invokeLater(new Runnable() {
            public void run() {
                new GUI34CalculadoraCombo(compacto);
            }
        });
    }
}

```

8.6 Lista 7 - Listeners - exercício 1

8.6.1 Exercício 1 - Abrir e salvar arquivo no editor

Contexto

Complete o editor da Lista 6, Exercício 1 (p. 18) com os eventos de abertura e salvamento. O padrão integra listener, JOptionPane, streams e exceções. O diálogo pode retornar null se o usuário cancelar; teste antes de abrir o recurso.

Abrir arquivo e carregar JTextArea

```
String nome = JOptionPane.showInputDialog(this, "Arquivo:");
if (nome == null || nome.trim().isEmpty()) return;
try (Scanner in = new Scanner(new FileInputStream(nome))) {
    StringBuilder texto = new StringBuilder();
    while (in.hasNextLine())
        texto.append(in.nextLine()).append('\n');
    area.setText(texto.toString());
} catch (FileNotFoundException e) {
    JOptionPane.showMessageDialog(this,
        "Arquivo não encontrado", "Erro",
        JOptionPane.ERROR_MESSAGE);
}
```

Salvar conteúdo da JTextArea

```
String texto = area.getText();
try (FileOutputStream out = new FileOutputStream(nome)) {
    out.write(texto.getBytes());
} catch (IOException e) {
    JOptionPane.showMessageDialog(this, "Falha ao salvar");
}
```

8.7 Lista 8 - Threads - exercícios 2, 6 e 7

8.7.1 Exercício 2 - Soma de matrizes em quatro quadrantes

Contexto

Leia duas matrizes inteiras $N \times N$. Quatro threads escrevem em quadrantes disjuntos de C; depois de iniciar todas, o programa chama join() antes de imprimir. O cálculo de meio também cobre N ímpar.
Conceitos: particionamento exclusivo, Runnable, start/join e ausência de corrida nas células.

Threads2SomaQuadrantes.java

```
import java.util.Scanner;

public class Threads2SomaQuadrantes {
    static class Worker implements Runnable {
        private final int[][] a, b, c;
        private final int li, lf, ci, cf;

        Worker(int[][] a, int[][] b, int[][] c,
            int li, int lf, int ci, int cf) {
            this.a = a;
            this.b = b;
            this.c = c;
            this.li = li;
            this.lf = lf;
            this.ci = ci;
            this.cf = cf;
        }

        @Override public void run() {
            for (int i = li; i < lf; i++)
                for (int j = ci; j < cf; j++)
                    c[i][j] = a[i][j] + b[i][j];
        }
    }

    static int[][] lerMatriz(Scanner sc, int n) {
        int[][] m = new int[n][n];
        for (int i = 0; i < n; i++)
            for (int j = 0; j < n; j++)
                m[i][j] = sc.nextInt();
        return m;
    }

    public static void main(String[] args)
        throws InterruptedException {
        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt();
        int[][] a = lerMatriz(sc, n);
        int[][] b = lerMatriz(sc, n);
        int[][] c = new int[n][n];
        int meio = (n + 1) / 2;

        Thread[] ts = {
            new Thread(new Worker(a, b, c, 0, meio, 0, meio)),
            new Thread(new Worker(a, b, c, 0, meio, meio, n)),
            new Thread(new Worker(a, b, c, meio, n, 0, meio)),
            new Thread(new Worker(a, b, c, meio, n, meio, n))
        };

        for (Thread t : ts) t.start();
    }
}
```



```

        for (Thread t : ts) t.join();

        for (int[] linha : c) {
            for (int x : linha) System.out.print(x + " ");
            System.out.println();
        }
    }
}

```

8.7.2 Exercício 6 - Troca de invokeLater por invokeAndWait

Contexto

Uma worker atualiza JProgressBar com invokeAndWait; só continua quando a alteração termina na EDT. A chamada não pode partir da própria EDT e exige tratamento de InterruptedException e InvocationTargetException.

Threads6InvokeAndWait.java

```

import java.lang.reflect.InvocationTargetException;
import javax.swing.*;

public class Threads6InvokeAndWait extends JFrame
    implements Runnable {
    private final JProgressBar barra = new JProgressBar(0, 10);
    private int contador;

    Threads6InvokeAndWait() {
        super("Thread com Swing");
        add(barra);
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        pack();
        setLocationRelativeTo(null);
        setVisible(true);
        new Thread(this, "progresso").start();
    }

    @Override public void run() {
        try {
            while (contador <= 10) {
                SwingUtilities.invokeLater(new Runnable() {
                    public void run() {
                        barra.setValue(contador++);
                    }
                });
                Thread.sleep(1000);
            }
        } catch (InterruptedException e) {
            Thread.currentThread().interrupt();
        } catch (InvocationTargetException e) {
            throw new RuntimeException(e.getCause());
        }
    }

    public static void main(String[] args) {
        SwingUtilities.invokeLater(new Runnable() {
            public void run() {
                new Threads6InvokeAndWait();
            }
        });
    }
}

```

8.7.3 Exercício 7 - Caça-níquel com três workers independentes

Contexto

Três campos giram entre 1 e 7 por quantidade aleatória de atualizações. O botão fica desabilitado até as três workers terminarem; um AtomicInteger identifica a última conclusão e agenda na EDT a verificação de vitória.

Conceitos: workers, estado atômico, atualização Swing na EDT e coordenação de término.

1. no clique, desabilite Jogar e crie três workers;
2. cada worker gera valores de 1 a 7, dorme e publica seu campo na EDT;
3. ao terminar, sinaliza conclusão; coordene com join ou contador sincronizado/atômico;
4. na EDT, compare os três valores finais, mostre o diálogo e reabilite o botão.

Cada worker altera um campo diferente, mas a decisão final depende dos três. O ponto central é coordenação de término e atualização Swing na EDT.

Threads7CacaNiquel.java

```

import java.awt.BorderLayout;
import java.awt.GridLayout;
import java.awt.event.ActionEvent;
import java.awt.event.ActionListener;
import java.util.concurrent.ThreadLocalRandom;
import java.util.concurrent.atomic.AtomicInteger;
import java.util.concurrent.atomic.AtomicIntegerArray;
import javax.swing.*;

public class Threads7CacaNiquel extends JFrame {
    private final JTextField[] campos = new JTextField[3];
    private final JButton jogar = new JButton("Jogar");
    private final AtomicIntegerArray valores = new AtomicIntegerArray(3);
    private final AtomicInteger restantes = new AtomicInteger();
}

```

```

Threads7CacaNiquel() {
    super("Caça-níquel");
    JPanel rolos = new JPanel(new GridLayout(1, 3, 8, 0));
    for (int i = 0; i < campos.length; i++) {
        campos[i] = new JTextField("1", 4);
        campos[i].setEditable(false);
        campos[i].setHorizontalAlignment(SwingConstants.CENTER);
        rolos.add(campos[i]);
    }

    jogar.addActionListener(new ActionListener() {
        public void actionPerformed(ActionEvent e) {
            iniciar();
        }
    });
    add(rolos, BorderLayout.CENTER);
    add(jogar, BorderLayout.SOUTH);
    setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
    pack();
    setLocationRelativeTo(null);
    setVisible(true);
}

private void iniciar() {
    jogar.setEnabled(false);
    restantes.set(3);
    for (int i = 0; i < 3; i++) {
        final int indice = i;
        new Thread(new Runnable() {
            public void run() {
                girar(indice);
            }
        }, "rolo-" + i).start();
    }
}

private void girar(final int indice) {
    int giros = ThreadLocalRandom.current().nextInt(1, 151);
    try {
        for (int i = 0; i < giros; i++) {
            final int valor = ThreadLocalRandom.current().nextInt(1, 8);
            valores.set(indice, valor);
            SwingUtilities.invokeLater(new Runnable() {
                public void run() {
                    campos[indice].setText(String.valueOf(valor));
                }
            });
            Thread.sleep(25);
        }
    } catch (InterruptedException e) {
        Thread.currentThread().interrupt();
    } finally {
        if (restantes.decrementAndGet() == 0) {
            SwingUtilities.invokeLater(new Runnable() {
                public void run() {
                    finalizar();
                }
            });
        }
    }
}

private void finalizar() {
    boolean venceu = valores.get(0) == valores.get(1)
        && valores.get(1) == valores.get(2);
    jogar.setEnabled(true);
    JOptionPane.showMessageDialog(this,
        venceu ? "Você venceu!" : "Você perdeu.");
}

public static void main(String[] args) {
    SwingUtilities.invokeLater(new Runnable() {
        public void run() {
            new Threads7CacaNiquel();
        }
    });
}
}

```

8.8 Lista 11 - Socket: jogo da velha em rede

Contexto completo

Dois jogadores executam clientes Swing separados. Um servidor aceita as duas conexões, atribui X e O, mantém o tabuleiro, valida a vez e a posição, detecta vitória/empate e transmite o estado. O cliente apenas mostra o tabuleiro e envia a casa clicada.

- **Servidor:** tabuleiro, símbolos, vez, validação, vitória e empate.
- **Cliente:** botões, status e envio de JOGAR posição.
- **Rede:** leitura bloqueante fora da EDT; tela atualizada com `invokeLater`.
- **Concorrência:** estado do jogo protegido ao atender os dois sockets.

Execução: inicie o servidor e depois duas instâncias do cliente.

Direção	Mensagem	Papel
servidor → cliente	SÍMBOLO X	papel local
servidor → clientes	TABULEIRO ...	estado completo
servidor → clientes	VEZ X	quem pode jogar
cliente → servidor	JOGAR 4	posição escolhida
servidor → cliente	ERRO ...	jogada recusada
servidor → clientes	FIM ...	vitória, empate ou desconexão

Atenção - regra de autoridade

O servidor valida e decide; o cliente apenas apresenta uma visão e envia intenções. Assim, um cliente defeituoso não cria jogadas impossíveis nem estados divergentes.

8.8.1 Servidor - autoridade sobre as regras

O método `Jogo.jogar` é sincronizado: valida, altera o estado e transmite a visão única do tabuleiro. Cada conexão é atendida por uma `Jogador` `extends Thread`.

ServidorJogoVelha.java

```
import java.io.*;
import java.net.*;
import java.util.Arrays;

public class ServidorJogoVelha {
    static class Jogo {
        private final char[] tabuleiro = new char[9];
        private Jogador x, o;
        private char vez = 'X';
        private boolean terminou;

        Jogo() {
            Arrays.fill(tabuleiro, '-');
        }

        synchronized void configurar(Jogador x, Jogador o) {
            this.x = x;
            this.o = o;
        }

        synchronized void iniciar() {
            x.enviar("SÍMBOLO X");
            o.enviar("SÍMBOLO O");
            broadcast("TABULEIRO " + new String(tabuleiro));
            broadcast("VEZ X");
        }

        synchronized void jogar(Jogador p, int posicao) {
            if (terminou) {
                p.enviar("ERRO Jogo encerrado");
                return;
            }
            if (p.simbolo != vez) {
                p.enviar("ERRO Aguarde sua vez");
                return;
            }
            if (posicao < 0 || posicao >= 9
                || tabuleiro[posicao] != '-') {
                p.enviar("ERRO Jogada inválida");
                return;
            }

            tabuleiro[posicao] = p.simbolo;
            broadcast("TABULEIRO " + new String(tabuleiro));
            if (venceu(p.simbolo)) {
                terminou = true;
                broadcast("FIM VENCEDOR " + p.simbolo);
            } else if (cheio()) {
                terminou = true;
                broadcast("FIM EMPATE");
            } else {
                vez = (vez == 'X') ? 'O' : 'X';
                broadcast("VEZ " + vez);
            }
        }

        synchronized void desconectou(Jogador p) {
            if (!terminou) {
                terminou = true;
                broadcast("FIM ADVERSARIO_DESCONECTOU");
            }
        }

        private boolean venceu(char s) {
            int[][] linhas = {
                {0,1,2}, {3,4,5}, {6,7,8},
                {0,3,6}, {1,4,7}, {2,5,8},
                {0,4,8}, {2,4,6}
            };
            for (int[] l : linhas) {
                if (tabuleiro[l[0]] == s
                    && tabuleiro[l[1]] == s
                    && tabuleiro[l[2]] == s) return true;
            }
            return false;
        }
    }
}
```

```

    private boolean cheio() {
        for (char c : tabuleiro)
            if (c == '-') return false;
        return true;
    }

    private void broadcast(String mensagem) {
        x.enviar(mensagem);
        o.enviar(mensagem);
    }
}

static class Jogador extends Thread {
    private final Socket socket;
    private final char simbolo;
    private final Jogo jogo;
    private final BufferedReader in;
    private final PrintWriter out;

    Jogador(Socket socket, char simbolo, Jogo jogo)
        throws IOException {
        this.socket = socket;
        this.simbolo = simbolo;
        this.jogo = jogo;
        in = new BufferedReader(new InputStreamReader(
            socket.getInputStream()));
        out = new PrintWriter(socket.getOutputStream(), true);
    }

    void enviar(String mensagem) {
        out.println(mensagem);
    }

    @Override public void run() {
        try (Socket s = socket;
            BufferedReader reader = in;
            PrintWriter writer = out) {
            String linha;
            while ((linha = reader.readLine()) != null) {
                if (linha.startsWith("JOGAR ")) {
                    try {
                        jogo.jogar(this,
                            Integer.parseInt(linha.substring(6)));
                    } catch (NumberFormatException e) {
                        enviar("ERRO Posição inválida");
                    }
                } else {
                    enviar("ERRO Comando desconhecido");
                }
            }
        } catch (IOException e) {
            System.out.println("Conexão encerrada: "
                + e.getMessage());
        } finally {
            jogo.desconectou(this);
        }
    }
}

public static void main(String[] args) throws Exception {
    try (ServerSocket servidor = new ServerSocket(5000)) {
        Jogo jogo = new Jogo();
        Jogador x = new Jogador(servidor.accept(), 'X', jogo);
        Jogador o = new Jogador(servidor.accept(), 'O', jogo);
        jogo.configurar(x, o);
        x.start();
        o.start();
        jogo.iniciar();
        x.join();
        o.join();
    }
}
}

```

8.8.2 Cliente Swing - interface e leitura em background

A thread de rede bloqueia em `readLine`; cada mensagem recebida é publicada na EDT. Os botões só ficam habilitados quando o servidor informa que é a vez do símbolo local.

ClienteJogoVelha.java

```

import java.awt.BorderLayout;
import java.awt.GridLayout;
import java.awt.event.ActionEvent;
import java.awt.event.ActionListener;
import java.io.*;
import java.net.Socket;
import javax.swing.*;

public class ClienteJogoVelha extends JFrame {
    private final JButton[] casas = new JButton[9];
    private final JLabel status = new JLabel(
        "Conectando...", SwingConstants.CENTER);
    private volatile PrintWriter out;
    private char simbolo, vez;
    private boolean terminou;
}

```

```

ClienteJogoVelha(final String host) {
    super("Jogo da Velha");
    JPanel grade = new JPanel(new GridLayout(3, 3));
    for (int i = 0; i < casas.Length; i++) {
        final int posicao = i;
        casas[i] = new JButton("");
        casas[i].setEnabled(false);
        casas[i].addActionListener(new ActionListener() {
            public void actionPerformed(ActionEvent e) {
                jogar(posicao);
            }
        });
        grade.add(casas[i]);
    }
    add(grade, BorderLayout.CENTER);
    add(status, BorderLayout.SOUTH);
    setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
    setSize(320, 340);
    setLocationRelativeTo(null);
    setVisible(true);
    new Thread(new Runnable() {
        public void run() {
            conectar(host);
        }
    }, "rede").start();
}

private void jogar(int posicao) {
    desabilitar();
    status.setText("Aguardando servidor...");
    if (out != null) out.println("JOGAR " + posicao);
}

private void conectar(String host) {
    boolean recebeuFim = false;
    try (Socket socket = new Socket(host, 5000);
        BufferedReader in = new BufferedReader(
            new InputStreamReader(socket.getInputStream()));
        PrintWriter writer = new PrintWriter(
            socket.getOutputStream(), true)) {
        out = writer;
        String linha;
        while ((linha = in.readLine()) != null) {
            if (linha.startsWith("FIM ")) recebeuFim = true;
            final String mensagem = linha;
            SwingUtilities.invokeLater(new Runnable() {
                public void run() {
                    processar(mensagem);
                }
            });
        }
    } catch (IOException e) {
        // A mesma atualização abaixo cobre erro e fim de stream.
    }
    if (!recebeuFim) avisarConexaoEncerrada();
}

private void avisarConexaoEncerrada() {
    SwingUtilities.invokeLater(new Runnable() {
        public void run() {
            desabilitar();
            status.setText("Conexão encerrada");
        }
    });
}

private void processar(String m) {
    if (m.startsWith("SIMBOLO ")) {
        simbolo = m.charAt(8);
        status.setText("Você é " + simbolo);
    } else if (m.startsWith("TABULEIRO ")) {
        String t = m.substring(10);
        for (int i = 0; i < 9; i++) {
            casas[i].setText(t.charAt(i) == '-'
                ? "" : String.valueOf(t.charAt(i)));
        }
    } else if (m.startsWith("VEZ ")) {
        vez = m.charAt(4);
        status.setText(vez == simbolo
            ? "Sua vez" : "Vez do adversário");
        atualizarBotoes();
    } else if (m.startsWith("ERRO ")) {
        status.setText(m.substring(5));
        atualizarBotoes();
    } else if (m.startsWith("FIM ")) {
        terminou = true;
        desabilitar();
        String fim = m.substring(4);
        String texto;
        if (fim.equals("EMPATE")) {
            texto = "Empate";
        } else if (fim.startsWith("VENCEDOR ")) {
            texto = fim.charAt(9) == simbolo
                ? "Você venceu" : "Você perdeu";
        } else {
            texto = "Adversário desconectou";
        }
        status.setText(texto);
        JOptionPane.showMessageDialog(this, texto);
    }
}

```

```

    }
    private void atualizarBotoes() {
        for (JButton b : casas) {
            b.setEnabled(!terminou && vez == simbolo
                && b.getText().isEmpty());
        }
    }
    private void desabilitar() {
        for (JButton b : casas) b.setEnabled(false);
    }
    public static void main(String[] args) {
        final String host = args.length == 0
            ? "localhost" : args[0];
        SwingUtilities.invokeLater(new Runnable() {
            public void run() {
                new ClienteJogoVelha(host);
            }
        });
    }
}

```

9 Checklist, resumo e índice reverso

Esta seção reúne o checklist final, o resumo dos assuntos e o índice reverso para localizar definições e padrões de código.

9.1 Checklist antes de entregar uma resposta de código

- | | |
|--|--|
| <ol style="list-style-type: none"> 1. Todas as classes, métodos, parênteses e chaves estão fechados? 2. O construtor não tem retorno e inicializa o estado válido? 3. A responsabilidade está na classe que conhece os dados? 4. Campos que precisam de proteção estão <code>private</code>? 5. Toda alteração preserva as invariantes? 6. A relação usa corretamente <code>extends</code>, <code>implements</code> ou <code>composição</code>? 7. Métodos de interface estão <code>public</code> e com assinatura exata? 8. <code>@Override</code> está onde há sobrescrita? 9. Strings e objetos são comparados com <code>equals</code> quando a intenção é conteúdo? 10. Índices e limites usam <code>< tamanho</code>, não <code><= tamanho</code>? 11. Toda <code>checked exception</code> foi tratada ou declarada? 12. A recuperação ocorre em uma fronteira capaz de informar ou repetir a operação? | <ol style="list-style-type: none"> 13. A coleção preserva índice, unicidade, chave e ordem exigidos? 14. Remoção durante percurso usa <code>Iterator.remove</code> ou estratégia segura? 15. A ordenação usa o mesmo critério para ordenar e buscar? 16. Listeners foram registrados na fonte correta? 17. Layouts aninhados e regiões do <code>BorderLayout</code> estão corretos? 18. Threads foram iniciadas com <code>start()</code>? 19. Resultados paralelos só são usados depois da coordenação necessária? 20. Estado compartilhado está protegido pelo mesmo monitor/lock? 21. Atualizações Swing ocorrem na EDT e tarefas longas fora dela? 22. Cliente e servidor seguem exatamente a mesma ordem de mensagens? 23. Streams e sockets são fechados em todos os caminhos? |
|--|--|

9.2 Resumo em uma frase por assunto

POO: distribua estado e responsabilidade em objetos.

Encapsulamento: exponha operações controladas e preserve invariantes.

Composição: represente colaboração por campos “tem um”.

Herança: use apenas para especialização verdadeira “é um”.

Polimorfismo: programe para o contrato e deixe o objeto escolher a implementação.

Interface: declare o que tipos diferentes precisam saber fazer.

Exceção: detecte na origem e recupere na fronteira adequada.

Collection: escolha pela propriedade - índice, unicidade, chave ou ordem.

GUI: componha layouts; trate ações por listeners.

Thread: defina a tarefa, proteja o compartilhado e coordene o término.

Socket: os dois lados precisam obedecer ao mesmo protocolo.

9.3 Índice reverso por dúvida ou palavra-chave

Dúvida / termo	Página
Classe x objeto x instância	5
Construtor, this e construtor vazio	5
Encapsulamento e invariante	5
Passagem de objetos por parâmetro	5
Sobrecarga (overload)	5
Composição ou herança?	6
Herança e super	6
Sobrescrita e método que executa	6
Polimorfismo e tipo geral	6
Classe abstrata ou interface?	7
extends OU implements?	6
enum para estado	17
try/catch/finally	7
Checked x unchecked	7
throw X throws	7
Entrada numérica inválida	8
Escolher List/Set/Map/Queue	8
ArrayList X Vector	8
RA repetido / busca por chave	15
Iterator X Enumeration	9
Remover durante laço	9
Comparable X Comparator	9

Dúvida / termo	Página
Montar janela Swing	9
BorderLayout/GridLayout/FlowLayout	9
pack() e janela pequena	10
Listener não funciona	10
Listener x Adapter	10
Abrir/salvar no JTextArea	21
start() X run()	10
sleep e ordem de saída	10
Resultado antes de join	11
Condição de corrida / synchronized	11
wait/notifyAll	11
GUI trava / regra da EDT	11
invokeLater X invokeAndWait	12
Escolher stream	12
Scanner e quebra de linha	12
File x conteúdo do arquivo	12
Cliente x servidor	12
Protocolo e bloqueio de rede	12
Soluções completas das listas	13

Boa prova

Em dúvida, volte ao contrato da classe: **o que o objeto sabe, o que deve fazer e qual estado precisa preservar**. Para concorrência e comunicação, acrescente: **quem executa, quem espera e qual protocolo coordena os participantes**.